

AdvPurge: A Robust Personalized Federated Learning Framework Against Backdoor Attack

Tu Huang and Na Ruan*

Shanghai Jiao Tong University, Shanghai, China
{sjtuht, naruan}@sjtu.edu.cn

Abstract. Federated Learning (FL), as a distributed model training paradigm, has garnered significant attention and practical application. Recently, increasing research efforts have focused on Personalized Federated Learning (PFL) as an effective solution to address data heterogeneity in FL systems. However, existing studies reveal that PFL remains vulnerable to stealthy yet harmful backdoor attacks. Furthermore, current federated learning (FL) algorithms designed to defend against backdoor attacks demonstrate significant degradation in model performance and substantial decline in defensive effectiveness when applied to personalized scenarios. To bridge this research gap, we propose a robust PFL framework against backdoor attacks. Our framework incorporates a three-tier defense mechanism: (1) Clients initially purify potential model backdoors through adversarial example generation; (2) An alternating training strategy for hierarchical models is employed to block backdoor attacks while generating personalized head models; (3) The server implements a trimmed aggregation mechanism to mitigate malicious client impacts. Comprehensive experiments on three benchmark datasets demonstrate the framework’s effectiveness, showing superior model performance in personalized scenarios while achieving up to 89% attack success suppression rate compared with eight state-of-the-art defense methods.

Keywords: Backdoor defense · Personalized federated learning

1 Introduction

Federated learning (FL) [1] emerged as a promising paradigm for collaborative model training while preserving data privacy. By enabling multiple clients to jointly learn a shared global model without sharing raw data, FL has found applications in healthcare, finance, and mobile computing. However, the inherent data heterogeneity across clients - where local data distributions are non-independent and identically distributed (non-IID) - poses fundamental challenges to conventional FL frameworks. A typical example occurs in cross-bank user behavior prediction : different financial institutions serve distinct user demographics with varying transaction patterns, spending habits, and risk profiles. The significant divergence in data distributions between banks makes traditional FL

* Corresponding author

approaches suboptimal, motivating the need for personalization mechanisms. To address this, personalized federated learning (PFL) [2], [3], [4], [5] has been proposed, allowing clients to develop customized models adapted to their local data characteristics.

While conventional federated learning (FL) and personalized federated learning (PFL) offer numerous advantages, their decentralized nature exposes systems to severe security threats from malicious participants launching backdoor attacks [6]. Backdoor attack [7] occurs when adversaries intentionally train local models using poisoned data containing specific trigger patterns, thereby implanting hidden functionalities in the compromised model. Unlike Byzantine attacks or poisoning attacks that aim to degrade the global model’s classification accuracy or prevent convergence, backdoor attacks pursue a stealthier objective: preserving model performance on clean samples while embedding malicious behaviors that activate exclusively on triggered inputs. This preservation of nominal functionality makes backdoor attacks particularly challenging to detect and mitigate.

Numerous studies [8], [9] have investigated defense mechanisms against backdoor attacks in conventional FL settings. However, existing defense strategies face critical limitations when applied to personalized federated learning scenarios. The non-IID nature of client data in PFL critically undermines existing backdoor defense mechanisms in two aspects. First, the substantial distribution divergence across clients severely degrades the performance of conventional defense systems and compromises the system’s functionality. Second, data heterogeneity inherently induces significant variations in client-specific model parameters, thereby invalidating the fundamental assumption of traditional defense strategies - which rely on detecting malicious clients through similarity analysis of gradient updates across participants.

Notably, recent work [10] revealed that state-of-the-art PFL architectures exhibit intrinsic robustness against backdoor attacks. That is to say, even without explicit defense mechanisms, existing PFL systems can inherently reduce attack success rates (ASR) of backdoor attempts. However, this inherent robustness manifests only when systems employ shallow convolutional neural networks (CNNs) with 3-4 layers. When deploying more complex neural architectures like ResNet, current PFL systems remain highly susceptible to backdoor attacks. The increased parameter space of deep networks enable adversaries to embed persistent triggers that survive personalization processes, thereby neutralizing the natural defensive benefits observed in simpler models.

Our contributions To defend against backdoor attacks in PFL, we propose **AdvPurge**, a novel PFL algorithm integrating hierarchical training, adversarial elimination and trimmed aggregation, which achieves both high robustness and superior model performance. In terms of robustness, AdvPurge enhances robustness through a base-head decoupling strategy. Specifically, only the model’s base component (a generic feature extractor unrelated to backdoor classification) is shared across clients, while the head component (task-specific classifier layers susceptible to backdoor triggers) remains locally stored on each client.

This architecture isolates backdoor-critical parameters (e.g., head layers) within individual clients, thereby blocking cross-client contamination of malicious updates. Furthermore, inspired by the behavioral similarity between adversarial noise and backdoor triggers on compromised models, AdvPurge empowers the client to eliminate latent backdoors by training the local model on adversarially perturbed samples. AdvPurge further enables the server to perform trimmed aggregation on base models, a method that conducts outlier trimming on each dimension of all uploaded base models, thereby achieving privacy-preserving model aggregation. As for the aspect of model performance, by enabling clients to maintain personalized head models, AdvPurge inherently adapts to non-IID data distributions—a pervasive challenge in PFL scenarios. This client-specific personalization preserves task-specific discriminative features while mitigating distributional conflicts, ultimately achieving state-of-the-art classification accuracy.

In summary, this paper makes the following contributions:

- **Three-tier Defense Architecture:** Propose AdvPurge, the first PFL framework integrating adversarial purification, base-head decoupling and trimmed aggregation to achieve 89% backdoor suppression rate while maintaining outstanding classification accuracy.
- **Adversarial-Guided Purification:** Develop a novel client-side defense mechanism that leverages adversarial examples to eliminate backdoor neurons through gradient inversion, requiring no prior knowledge of trigger patterns.
- **Parameter Isolation Strategy:** Design a dimension-wise trimmed aggregation algorithm that isolates malicious parameters through layer-wise outlier elimination, effectively neutralizing most of attack attempts under non-IID settings.

2 Background and Related Work

2.1 Personalized federated learning

Prior to detailing our methodology, we formally define federated learning (FL) and personalized federated learning (PFL).

Conventional federated learning Federated learning is a decentralized algorithm enabling multiple data owners to collaboratively train machine learning models without sharing their local data. There exist n clients $\{C_i\}_{i=1}^n$, each holding a private dataset D_i . A central server coordinates model aggregation without accessing raw client data. The optimization object for conventional federated learning is :

$$\min_{\theta_g} \sum_{i=1}^n p_i \mathcal{L}(f(x_i; \theta_g), y_i) \quad (1)$$

, where $p_i = \frac{|D_i|}{\sum_{j=1}^n |D_j|}$, and $\mathcal{L}(\cdot, \cdot)$ denotes loss function, such as the cross entropy loss and K-L divergence loss.

Personalized federated learning Unlike conventional federated learning that trains a single global model, personalized federated learning (PFL) emphasizes constructing customized models for heterogeneous clients through different algorithm designs. Current PFL approaches can be categorized into full model-sharing method and partial model-sharing method [10]. The full model-sharing method [11], [12] allows clients to maintain their own local personalized model while exchanging global parameters with other clients. The optimization object for full model-sharing method is

$$\min_{\theta_g, \{\theta_i\}_{i=1}^n} \sum_{i=1}^n [p_i \mathcal{L}(f(x_i; \theta_i), y_i) + \lambda \mathcal{H}(\theta_i, \theta_g)] \quad (2)$$

, where θ_g is the global model parameter shared among clients, θ_i is the local personalized model parameter kept by client i , $\mathcal{H}(\cdot, \cdot)$ is the regularizer of similarity between θ_i and θ_g , and λ is the regularization coefficient kept by each client to control the degree of regularization. In each round, clients receive θ_g and use it to train their local θ_i . θ_g is then further trained on local datasets and uploaded to the server to aggregate. Therefore, full model sharing through θ_g enables knowledge transfer from each client's local dataset to other clients' local models.

For partial model-sharing method [3], [4], the classification model is divided into two models, the global-part model is shared among clients, and the personalized-part model is kept local by each client. The optimization object for partial model-sharing method is

$$\min_{\theta_g, \{\theta_{p_i}\}_{i=1}^n} \sum_{i=1}^n p_i \mathcal{L}(f(x_i; \theta_{p_i}, \theta_g), y_i) \quad (3)$$

, where the θ_{p_i} is the personalized-part model kept by client i , and θ_g is the global-part model shared among clients.

2.2 Backdoor attack and defense in federated learning

Backdoor attacks in FL have been identified by researchers as a critical latent security threat to FL systems [13]. The primary goal of such attacks is to implant backdoors into the global model by manipulating local models within the FL framework. The adversary's optimization objective in a backdoor attack is:

$$\min_{\theta^*} \sum_{i \in D_c} \mathcal{L}(x_i, y_i; \theta) + \sum_{i \in D_b} \mathcal{L}(x_i + \delta, y_t; \theta), \quad (4)$$

where D_c represents the clean dataset used by the adversary to train the model on the legitimate classification task, while D_b denotes the dataset employed to train the backdoor task. By injecting a trigger pattern δ into original samples, the adversary ensures that the model misclassifies the triggered samples into the predefined target label y_t .

To address this critical security threat in Federated Learning (FL) systems, researchers have proposed various defense mechanisms to mitigate backdoor attacks. These methods can be primarily categorized into two major classes: backdoor attack detection and backdoor attack mitigation. The objective of backdoor attack detection is to identify whether local models trained by clients contain backdoors. For instance, Li et al.[14] proposed a detection technique based on out-of-distribution (OOD) data. the server injects a probe task into the global model using OOD data, then it can accurately detect the presence of backdoors in uploaded models by evaluating the probe task. Ma et al.[15] discovered the correlation between topological properties and the neurons’ status. Based on this, they trained a binary classifier for the network topological structure to detect whether there were backdoors in the model in federated learning. However, such backdoor detection approaches can only determine whether a backdoor exists in the network but cannot remove it. Several research efforts have focused on backdoor elimination in federated systems. Zhang et al.[16] proposed a backdoor trigger reverse-engineering method to enhance the robustness of federated learning systems against backdoor attacks. Tan et al.[17] developed a prototype-sharing based federated learning framework that mitigates the impact of adversary’s prototypes through aggregation of highly similar prototypes. However, these studies primarily address conventional federated learning scenarios, without considering PFL system with non-IID data distributions.

2.3 Adversarial Example

Adversarial examples [18] are a type of attack designed to deceive trained models into making incorrect predictions. Formally, we denote by $f : \mathcal{X} \rightarrow \{1, \dots, n\}$ a classifier. For an input $x \in \mathcal{X}$ and a label $l = f(x)$, we call a vector r an adversarial noise if it satisfies:

$$\|r\| \leq \epsilon, f(x + r) \neq l,$$

Adversarial example attacks and backdoor attacks both aim to induce erroneous model outputs, yet they differ fundamentally in their mechanisms. Adversarial examples typically operate during the post-training phase of models, whereas backdoor attacks are generally implemented during the training phase. Furthermore, the misclassification caused by adversarial examples often yields unpredictable errors, while backdoor attacks produce errors specifically predetermined by attackers. However, existing research has revealed intriguing connections between these two attack paradigms. For instance, Mu et al. [19] demonstrated that in backdoored models, both adversarial examples and backdoor triggers activate similar neural pathways. Yin et al. [20] discovered that adversarial samples generated from backdoor-triggered inputs in compromised models tend to be predicted as their genuine labels. These findings provide novel insights for backdoor defense methodologies.

3 Problem Definition and Threat Model

3.1 System model

Suppose we have N clients in our PFL system, each client processes a dataset $D_i = (X_i, Y_i) = \{(x_i^1, y_i^1), (x_i^2, y_i^2), \dots, (x_i^{N_i}, y_i^{N_i})\}$, where $(x_i^j, y_i^j) \in \mathcal{X} \times \mathcal{Y}$ is a pair of training sample and its corresponding label, and N_i is the number of samples owned by the i -th client. Note in our personalized scenario, the data for each client is non-independent and identically distributed (non-iid), so we denote the data for the i -th client is generated by a distribution $(X_i, Y_i) \sim D_i$. The learning model for each client i is $f_i : \mathcal{X} \rightarrow \mathcal{Y}$, which maps inputs $x \in \mathcal{X}$ to its predict label $f_i(x) \in \mathcal{Y}$, which we hope it can be as close as possible to the real label $y \in \mathcal{Y}$.

To be specific, to better accommodate non-IID data distributions in personalized federated learning, we adopt a model-splitting framework similar to prior PFL approaches [3], [4]. In this architecture, all clients collaboratively train and share a common base model $B : \mathcal{X} \rightarrow \mathbb{R}^k$, which serves as a universal feature extractor that produce a low-dimensional k -dimensional feature representation from the data samples. This base model B captures generic data representations without incorporating task-specific classification layers. In m -th round of the PFL system, client i first train the base model B_m using their local data and transmit the updated $\theta_{B_m}^i$ parameters to the server. Subsequently, the server aggregates the collected client models, and applies privacy-preserving backdoor mitigation procedures before broadcasting the refined global base model parameter $\theta_{B_{m+1}}$ back to all clients. Upon receiving the updated global base model parameter $\theta_{B_{m+1}}$, each client directly replaces their local base model parameter with the latest version, thereby ensuring synchronized B parameters across the federation while preserving personalization capabilities through client-specific head models.

Subsequently, The classification functionality is instead delegated to personalized head models $H_i : \mathbb{R}^k \rightarrow \mathcal{Y}$ maintained locally by each client i . In each round of PFL, the client also uses its own local data to train its own personalized head model H_i . Since label distributions vary significantly across clients due to non-IID data in PFL system, this decoupled design enables the personalized head models to better align with their respective local label spaces. Consequently, The whole model for client i is the combination of the global base model and the personalized head model: $f_i(x) = (B \circ H_i)$. The jointly trained framework achieves enhanced prediction accuracy when inferring on each clients unique data distribution, as both the globally shared feature extractor and client-specific classifiers are optimized for their designated roles.

3.2 Threat Model

Adversary's capabilities We assume that the adversary controls a subset of clients in the federated learning system, referred to as malicious clients. During the local training phase of each federated learning round, the malicious clients

add backdoor trigger to a portion of their local training data when training their local model. This aims to ensure that the global model produces outputs specified by the adversary when encountering inputs embedded with backdoor triggers.

Adversary’s knowledge The adversary only has access to and control over their local training datasets, the global model parameters transmitted from the server in each round, and the training process of the malicious clients. However, the adversary cannot access the model parameters or training data of other benign clients, nor does it possess knowledge about the servers model aggregation process.

Adversary’s goal The adversarys goals are two-fold: 1) The aggregated global model should output the desired decision output specified by the adversary when inputs contain backdoor triggers, while maintaining correct predictions on non-triggered samples. 2) The backdoor attack should achieve a high attack success rate (ASR) without significantly degrading the models accuracy on clean samples.

3.3 Defense Goal

Similar to previous federated learning backdoor defense work, we define the capability of defender server to be:

- The defending server has no prior knowledge of the adversarys backdoor trigger patterns (e.g., shape, location, or semantic features). It can only access client-submitted model parameters and perform model aggregation and backdoor mitigation according to our proposed algorithms.
- The server does not possess any auxiliary data. It has no access to additional client-specific training information, ensuring compliance with federated learnings privacy principles.

Furthermore, we define the defense goal as ensuring that the proposed method can significantly suppresses the attack success rate (ASR) on backdoored samples, effectively neutralizing adversarial triggers. It can also maintains high model accuracy (ACC) on clean samples, ensuring no degradation in the global models primary task performance.

4 The Proposed AdvPurge Method

As shown in figure 1, AdvPurge is composed of three modules. In the first SAT(**A**dversarial **B**ackdoor **P**urge) module, clients eliminate possible backdoor in local model with the help of adversarial examples. In the second ST(**B**ase-**H**ead **S**plit **T**raining) module, the client separately trains the global base model and the personalized head model to better adapt to the data non-iid distribution

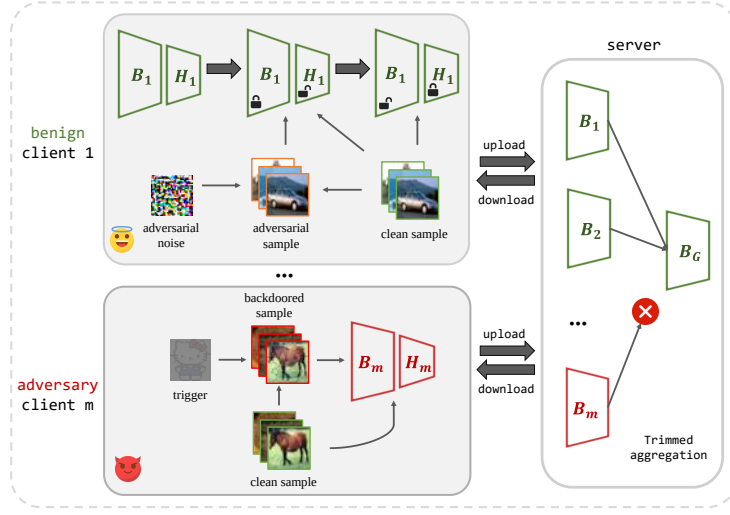


Fig. 1: An overview of AdvPurge Method. At each round, benign clients generate adversarial samples, and perform backdoor removal on their local models, and conduct stratified training with clean samples. Meanwhile, adversarial clients inject backdoor triggers into the model. Then, all selected clients upload their base models to the server, and the server performs trimmed aggregation.

in PFL and reduce the impact of the backdoor attacker on the local model. In the third TA(**T**rimmed **A**ggregation) module, the server analyzes the model uploaded by the client, and eliminates the possible backdoors in the global model by trimming the suspicious model parameters. The complete pseudocode of the AdvPurge algorithm can be found in Algorithm 1.

4.1 Adversarial Backdoor Purge

At the beginning of the m -th round, the server sends the current global base model parameters θ_B^m to all clients. Then, the α ratio of users is randomly selected to participate in the current round of PFL training. We assume client i is selected in round m . Upon receiving the parameters, client i replaces its local base model parameters with θ_B^m . Since possible backdoor may still exist in θ_B^m , we first use adversarial example to eliminate residual backdoored neuronal activations. Our adversarial backdoor purge method is inspired by previous work [19], which reveals an important property of backdoored models: When performing adversarial attacks on backdoored model $F_{backdoored}$, the misclassified labels $\tilde{y}$ induced by adversarial examples $\tilde{x}$ exhibit a high degree of consistency with the backdoor target label y^t caused by backdoor samples x^t . This means for backdoored model, backdoored samples and adversarial examples activates very similar network neuron. Based on this finding, we can eliminate residual backdoor in global base model θ_B^m .

Algorithm 1. AdvPurge Method

Input: Global base model θ_{B_m} , client datasets D_i , trimming ratio β
Output: Updated global model $\theta_{B_{m+1}}$

```

1 //----- 1. Adversarial Backdoor Purge Module -----
2 foreach client  $i \in \text{selected clients}$  do
3   Generate adversarial examples  $\tilde{D}_i$  using PGD on  $D_i$ ;
4   Purge backdoor via  $\theta'_{H_i} \leftarrow \arg \min_{\theta_H} \mathcal{L}(f(\tilde{x}; \theta_{B_m}, \theta_H), y)$ ; // Eq (5)
5 //----- 2. Base-Head Split Training Module -----
6 foreach client  $i \in \text{selected clients}$  do
7    $\theta_{B_m} \leftarrow \arg \min_{\theta_B} \sum \mathcal{L}(f(x; \theta_B, \theta_{H_{m-1}}), y)$ ; // Eq (7)
8    $\theta_{H_m} \leftarrow \arg \min_{\theta_H} \sum \mathcal{L}(f(x; \theta_{B_m}, \theta_H), y)$ ; // Eq (8)
9 //----- 3. Trimmed Aggregation Module -----
10  $\theta_{B_{m+1}} \leftarrow \mathbf{0}_d$ ;
11 for  $j = 1$  to  $d$  do
12    $\mathcal{S}_j \leftarrow \{\theta_{H_1}[j], \dots, \theta_{H_n}[j]\}$ ;
13   Sort  $\mathcal{S}_j$  to get ordered sequence  $\theta_{(1)j} \leq \dots \leq \theta_{(n)j}$ ;
14    $k_{\text{low}} \leftarrow \lceil \beta n \rceil + 1$ ,  $k_{\text{high}} \leftarrow n - \lfloor \beta n \rfloor$ ;
15    $\mathcal{T}_j \leftarrow \{\theta_{(k)j} \mid k_{\text{low}} \leq k \leq k_{\text{high}}\}$ ;
16    $\theta_{B_{m+1}}[j] \leftarrow \frac{1}{|\mathcal{T}_j|} \sum \mathcal{T}_j$ ;
17 return  $\theta_{B_{m+1}}$ ;

```

Specifically, we first generate adversarial examples $\tilde{x}$ using data $x \in D_i$ from the training dataset of client i , where the adversarial example generation method employs the Projected Gradient Descent (PGD)[21] algorithm:

$$\begin{aligned} \tilde{x}^{(t+1)} &= \text{Proj}_\epsilon \left(\tilde{x}^{(t)} + \alpha \cdot \text{sign} \left(\nabla_x \mathcal{L}(f(\tilde{x}^{(t)}; \theta_B^m, \theta_{H_i}), y) \right) \right), \\ t &= 1, \dots, n, \quad \tilde{x}^{(1)} = x, \quad \tilde{x} = \tilde{x}^{(n+1)} \end{aligned} \quad (5)$$

, where $\text{Proj}_\epsilon(\tilde{x}) = \text{clip}(\tilde{x}, x - \epsilon, x + \epsilon)$, α is the hyper-parameter regulating the scale of noise each iteration, ϵ is the hyper-parameter regulating the scale of noise added to the auxiliary data, and n is the iteration round.

After generating adversarial example for $\forall x \in D_i$, we obtain a adversarial auxiliary dataset $\tilde{D}_i = \{(\tilde{x}_i, y_i)\}_{i=1}^{|D_i|}$. Then, we can purge the residual backdoor in the model $f(\theta_{B_m}, \theta_{H_i})$ with $\tilde{D}_i$, which can be formalized as the following formula:

$$\theta'_{H_i} = \arg \min_{\theta_H} \mathbb{E}_{(\tilde{x}_i, y_i) \in \tilde{D}_i} [\mathcal{L}(f(\tilde{x}_i; \theta_B^m, \theta_H), y_i)] \quad (6)$$

4.2 Base-Head Split Training

After eliminating the backdoor of his model using adversarial samples, user i continues to participate in federated learning with base-head split training. He optimizes both the base model and personalized head model according to the

following objective:

$$\theta_{B_m} = \arg \min_{\theta_B} \frac{1}{N_i} \sum_{j=0}^{N_i} \mathcal{L}(f(x_j; \theta_B, \theta_{H_{m-1}}), y_j) \quad (7)$$

$$\theta_{H_m} = \arg \min_{\theta_H} \frac{1}{N_i} \sum_{j=0}^{N_i} \mathcal{L}(f(x_j; \theta_{B_m}, \theta_H), y_j) \quad (8)$$

Notably, the optimization processes for parameters θ_B (base model) and θ_H (personalized head models) in Equations 7 and 8 follow an alternating training process. Specifically, In the Head Model Phase, the client fix θ_B and optimize θ_H using local client data. And in base model phase, the client fix θ_H and optimize θ_B through server-mediated aggregation. This alternating optimization strategy ensures: The base model converges to extract **domain-agnostic feature representations** via cross-client aggregated training on $(X, Y) \sim \bigcup_{i=1}^n \mathcal{D}_i$, where $\mathcal{D}_i$ means the data distribution for client i . And each personalized head model specializes in **client-specific classification** via local training on $(X, Y) \sim \mathcal{D}_i$. It should be emphasized that while clients optimize their local models, the central server simultaneously trains a head model H_{aux} using its auxiliary dataset D_{aux} according to Equation 7, which will be strategically employed for backdoor mitigation in the subsequent phase.

4.3 Trimmed Aggregation

Upon completion of the m -th round training at client i , the trained base model B_m^i is transmitted to the central server. After receiving base models from all selected clients this round, the server need to initiate global base model aggregation. However, although base-head decoupled training is implemented, backdoored neuronal activations induced by trigger-embedded backdoor samples still exist in the base model uploaded by the adversary, which may do great harm to our global base model, necessitating our trimmed aggregation method.

As shown in part 3 in Algorithm 1, the core idea of our trimmed aggregation algorithm design is that, due to the design of our Base-head split training algorithm, the base model from all benign clients should be a common feature extractor. This means for all benign clients, the parameters of the base model they upload should be very close in all dimensions. However, for the adversary, since they used the backdoored samples with trigger to train the model, the base model parameters uploaded by adversary should have activated some different neurons from the model uploaded by benign client, which means the base model parameters uploaded by adversary greatly differs the base model parameters uploaded by the benign clients in some dimensions. Thus for all the base models received, the server can trim the largest β ratio and the smallest β ratio of the model parameters for each dimension, then average the remaining model parameters.

5 Experiment

5.1 Experiment Setting

Dataset and Models We employed three datasets in total: the MNIST, FashionMNIST, and CIFAR10 datasets. Both MNIST and FashionMNIST are 10-class classification datasets containing 70,000 grayscale images each with a resolution of 28×28 pixels. The CIFAR10 dataset is also a 10-class classification dataset, consisting of 60,000 color images with a resolution of 32×32 pixels. In terms of model selection, following prior work [10], [17], we adopted a CNN architecture comprising two convolutional layers and two fully connected layers for all three datasets. Additionally, we implemented the ResNet18 network specifically for the CIFAR10 dataset. We denote the experiment using a simple CNN model for CIFAR10 classification as Cifar10(C), and the experiment using ResNet18 for CIFAR10 classification as Cifar10(R).

Training Details For all datasets, we employed a Dirichlet distribution with $\alpha=0.5$ to partition the data into 40 clients, among which 5 adversary-controlled clients were designated to launch backdoor attacks. The malicious clients contained 20% poisoned samples in their local data. In each round, the server randomly selected 25% (i.e., 10 clients) to participate in training. The federated learning process was conducted over 1000 rounds across all datasets.

Attack Setting We configured all malicious clients to employ identical trigger patterns and share the same target label for backdoor attacks, with the adversarial target set to $y_t = 0$ in our experiments. Three distinct backdoor trigger types were implemented:

- **BadNet** : A 3×3 -pixel square trigger embedded in the bottom-right corner of images.
- **Blend** : A Hello Kitty watermark blended with original images at a mixing ratio of $\alpha=0.2$.
- **Sig** : A sinusoidal signal trigger with amplitude $A=30$ (within pixel range $[0, 255]$).

Visual examples of these triggers are demonstrated in Figure 2. Unlike previous studies where adversaries conduct backdoor attacks either through single-round injection [16] or delayed activation after model stabilization [17], our experimental framework enforces a more challenging scenario in which malicious clients persistently inject backdoor triggers throughout all training phases. This comprehensive attack strategy rigorously demonstrates the superior robustness of our method against continuous adversarial manipulation.

Baselines To establish the effectiveness of our AdvPurge method in defending against backdoor attacks, we compared AdvPurge with: 1) the baseline FedAvg[1]

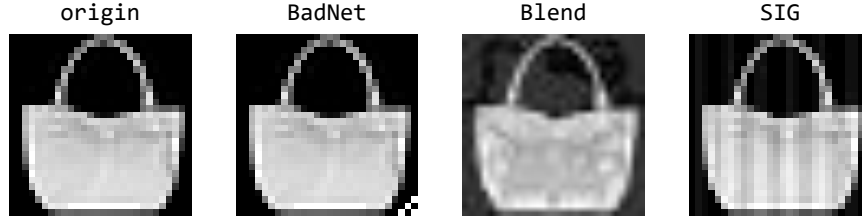


Fig. 2: Visualization of Different Backdoor Trigger Patterns

algorithm; 2) four state-of-the-art defense methods (Median[22], Trimmed-mean[22], Bulyan[23], and Multi-krum[24]); 3) FedRep[3] and FedProto[5] algorithms - as prior work [10] has indicated that mainstream personalized federated learning (PFL) algorithms exhibit inherent robustness against backdoor attacks; and 4) the latest FedPD[17] algorithm for comprehensive comparison.

Evaluation Metrics We employed classification accuracy (ACC) and attack success rate (ASR) as evaluation metrics. ACC measures the classification accuracy of personalized models trained by benign clients on clean samples without backdoor triggers, while ASR quantifies the proportion of clean samples embedded with identical trigger patterns (used by adversaries during backdoor attack training) that are misclassified into the adversary-specified target label y_t by these benign client-trained personalized models.

5.2 Defenses effectiveness against backdoor and comparison with SOTA

Table 1: Performance Comparison of Different Methods

Methods	MNIST		FMNIST		Cifar10 (C)		Cifar10 (R)	
	ACC	ASR	ACC	ASR	ACC	ASR	ACC	ASR
FedAvg	98.52	97.38	89.76	98.25	66.15	96.2	82.14	99.99
Median	98.9	0.14	89.35	6.19	58.23	6.75	21.55	75.29
Trimmed-mean	99.04	0.1	90.36	30.56	64.06	16.79	82.5	98.88
Bulyan	98.97	0.11	88.74	2.36	57.36	6.87	24.35	23.55
Multi-krum	99.2	2.03	90.23	2.8	65.19	93.06	82.59	98.53
FedRep	98.77	0.3	92.1	2.21	74.55	9.09	88.25	56.33
FedProto	98.04	0.31	92.17	0.91	69.85	4.21	70.57	4.26
FedPD	98.32	0.23	91.49	1.04	69.17	3.85	69.64	4.69
AdvPurge(Ours)	98.6	0.18	92.89	0.84	73.41	2.95	86.27	11.14

We conducted a comprehensive comparative analysis between AdvPurge and baseline methods in defending against backdoor attacks with backdoor trigger, with detailed results presented in Table 1. On the MNIST dataset, all defense methods except FedAvg demonstrated high classification accuracy and effective backdoor resistance, which can be attributed to the dataset’s inherent simplicity. However, in the more complex FMNIST environment, conventional federated learning defenses including FedAvg, Median, and Trimmed Mean showed significant vulnerabilities to backdoor attacks. Notably, the non-iid data distribution characteristic of Personalized Federated Learning (PFL) configurations induced an approximate 3% accuracy degradation in these conventional defense mechanisms, while AdvPurge achieves the highest ACC and lowest ASR.

For the Cifar10 benchmark, we implemented two distinct model architectures: simple CNN and ResNet18. With the simple CNN structure, AdvPurge achieved the lowest Attack Success Rate (ASR) of 0.72% while maintaining competitive classification accuracy (73.41%), merely 1% below the top-performing FedRep algorithm. The ResNet18 experiments exposed critical limitations of existing approaches: conventional federated learning defenses either suffered from unacceptably low accuracy ($ACC < 65\%$) or exhibited vulnerability to backdoor attacks ($ASR > 95\%$). While FedProto and FedPD demonstrated low ASR values (3.21% and 2.87% respectively), their subpar classification accuracy (70.57% and 69.64%) - even lower than FedAvg’s 82.14% - suggests these low ASR metrics stem from model underfitting rather than genuine defensive capability. AdvPurge emerged as the singular solution achieving optimal balance, delivering superior classification accuracy (86.27%) alongside robust backdoor resistance ($ASR = 11.14\%$), thereby establishing a new state-of-the-art in secure personalized federated learning frameworks.

5.3 Impact of Number of Adversary on Backdoor Attack Defense Effectiveness

We conducted experiments to investigate the impact of varying numbers of attackers on the results in Figure 3. When no attackers were present, none of the algorithms suffered from backdoor attacks, with the Attack Success Rate (ASR) approximating the classification error rate divided by the total classification attempts. FedAvg became fully vulnerable to backdoor attacks even with a single attacker, while FedPD exhibited minimal ASR fluctuations but consistently low accuracy (ACC), indicating poor feature learning. FedRep achieved high ACC across all scenarios but only resisted attacks with very few adversaries. In contrast, AdvPurge, despite a slightly lower ACC (1% reduction compared to FedRep), demonstrated robust defense against backdoor attacks even when half of the participants were malicious, highlighting its superior adversarial resilience.

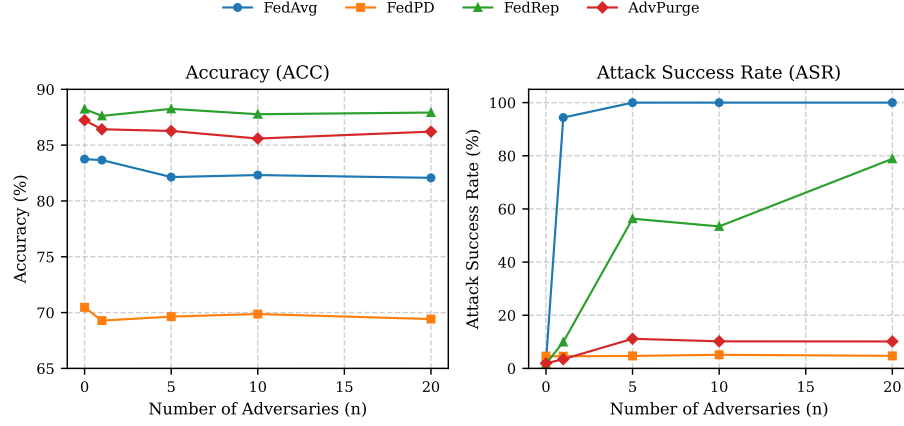


Fig. 3: Impact of Number of Adversary on Backdoor Attack Defense Effectiveness

Table 2: Influence of Personalization Degree on Defense Efficacy Against Backdoor Attacks

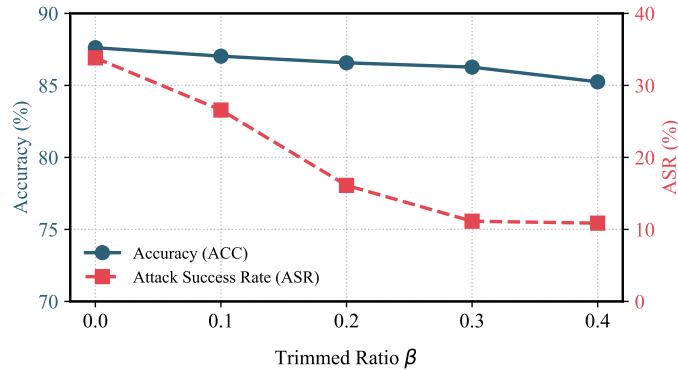
Method	$\alpha = 0.2$		$\alpha = 0.5$		$\alpha = 0.8$		iid	
	ACC	ASR	ACC	ASR	ACC	ASR	ACC	ASR
FedAvg	86.19	97.34	82.14	99.99	82.58	94.36	84.33	100
FedProto	82.39	3.91	70.57	4.26	63.99	3.6	46.55	5.58
FedPD	82.01	4.14	69.64	4.69	62.38	4.25	46.68	4.91
FedRep	92.22	10.52	88.25	56.33	86.4	81.77	83.27	97.73
AdvPurge	90.36	12.84	86.27	11.14	84.17	13.7	82.32	15.09

5.4 Influence of Personalization Degree on Defense Efficacy Against Backdoor Attacks

We conducted experiments to investigate the impact of personalization levels in user data distributions on the experimental results (as shown in Table 2), where a smaller α indicates higher personalization among user data (iid denotes identical distributions across all users). The results demonstrate that FedAvg suffers severe backdoor attacks under all data distribution scenarios, while other algorithms exhibit significant accuracy (ACC) improvements as data personalization increases. Notably, only the AdvPurge algorithm successfully resists backdoor attacks while achieving exceptionally high ACC, maintaining robust defense effectiveness even under varying degrees of data personalization.

Table 3: Effects of Trigger Pattern Variations on Backdoor Defense Robustness

Trigger Type	MNIST		FMNIST		Cifar10(C)		Cifar10(R)	
	ACC	ASR	ACC	ASR	ACC	ASR	ACC	ASR
BadNet	98.6	0.18	92.89	0.84	73.41	2.95	86.27	11.14
Blend	98.47	0.23	93.11	1.55	72.92	6.07	85.54	12.03
SIG	98.37	0.24	92.73	1.13	72.83	2.86	86.08	10.48

Fig. 4: Effects of Trimmed Ratio β on Defense Efficacy Against Backdoor Attacks

5.5 Effects of Trigger Pattern Variations on Backdoor Defense Robustness

We also investigated the impact of different backdoor attack triggers on the defensive efficacy of AdvPurge, with experimental results summarized in Table 3. The findings demonstrate that AdvPurge successfully mitigated all tested backdoor attack variants, while maintaining consistently high model accuracy (ACC) across different trigger types, with minimal performance deviation. Notably, the attack success rate (ASR) reached its peak when employing the Blend trigger, followed by BadNet, whereas the SIG trigger exhibited relatively lower threat potency under this defense framework.

5.6 Effects of Trimmed Ratio on Defense Efficacy Against Backdoor Attacks

We further investigated the impact of different trimmed ratios β on the defense performance of the model, with experimental results detailed in Figure 4. It can be observed that when $\beta = 0$, AdvPurge still demonstrates a certain level of defense effectiveness against backdoor attacks. As β gradually increases, the enhanced trimming proportion during the base model aggregation process leads to a substantial reduction in the Attack Success Rate (ASR), while only causing a

marginal decline in model accuracy (ACC). This indicates that our method effectively suppresses malicious backdoor parameters through trimmed aggregation without significantly compromising the model’s core functionality.

6 Conclusion

In this paper, We propose AdvPurge, a robust personalized federated learning framework against backdoor attack integrating three defenses: adversarial purification (eliminating backdoors via client-side adversarial training), base-head decoupling (isolating shared features from personalized classifiers), and trimmed aggregation (server-side parameter filtering) to suppress backdoor attacks under non-IID data. Experiments on MNIST, FashionMNIST, and CIFAR10 demonstrate AdvPurge achieves 89% attack suppression while maintaining high accuracy (e.g., 86.21% on CIFAR10), outperforming eight baselines, including FedRep and FedPD, especially in deep architectures like ResNet. Future work will explore adaptive defenses against dynamic triggers, and computational optimization.

References

1. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *Artificial intelligence and statistics*, pp. 1273–1282, PMLR, 2017.
2. A. Fallah, A. Mokhtari, and A. Ozdaglar, “Personalized federated learning with theoretical guarantees: A model-agnostic meta-learning approach,” *Advances in neural information processing systems*, vol. 33, pp. 3557–3568, 2020.
3. L. Collins, H. Hassani, A. Mokhtari, and S. Shakkottai, “Exploiting shared representations for personalized federated learning,” in *International conference on machine learning*, pp. 2089–2099, PMLR, 2021.
4. X. Li, M. Jiang, X. Zhang, M. Kamp, and Q. Dou, “Fedbn: Federated learning on non-iid features via local batch normalization,” *arXiv preprint arXiv:2102.07623*, 2021.
5. Y. Tan, G. Long, L. Liu, T. Zhou, Q. Lu, J. Jiang, and C. Zhang, “Fedproto: Federated prototype learning across heterogeneous clients,” in *Proceedings of the AAAI conference on artificial intelligence*, vol. 36, pp. 8432–8440, 2022.
6. E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, “How to backdoor federated learning,” in *International conference on artificial intelligence and statistics*, pp. 2938–2948, PMLR, 2020.
7. T. Gu, K. Liu, B. Dolan-Gavitt, and S. Garg, “Badnets: Evaluating backdooring attacks on deep neural networks,” *IEEE Access*, vol. 7, pp. 47230–47244, 2019.
8. J. Zhang, C. Zhu, C. Ge, C. Ma, Y. Zhao, X. Sun, and B. Chen, “Badcleaner: defending backdoor attacks in federated learning via attention-based multi-teacher distillation,” *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 5, pp. 4559–4573, 2024.
9. J. Zhang, C. Zhu, X. Sun, C. Ge, B. Chen, W. Susilo, and S. Yu, “Flpurifier: backdoor defense in federated learning via decoupled contrastive training,” *IEEE Transactions on Information Forensics and Security*, 2024.

10. Z. Qin, L. Yao, D. Chen, Y. Li, B. Ding, and M. Cheng, "Revisiting personalized federated learning: Robustness against backdoor attacks," in *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 4743–4755, 2023.
11. O. Marfoq, G. Neglia, A. Bellet, L. Kameni, and R. Vidal, "Federated multi-task learning under a mixture of distributions," *Advances in Neural Information Processing Systems*, vol. 34, pp. 15434–15447, 2021.
12. C. T. Dinh, N. Tran, and J. Nguyen, "Personalized federated learning with moreau envelopes," *Advances in neural information processing systems*, vol. 33, pp. 21394–21405, 2020.
13. T. D. Nguyen, T. Nguyen, P. Le Nguyen, H. H. Pham, K. D. Doan, and K.-S. Wong, "Backdoor attacks and defenses in federated learning: Survey, challenges and future research directions," *Engineering Applications of Artificial Intelligence*, vol. 127, p. 107166, 2024.
14. S. Li and Y. Dai, "{BackdoorIndicator}: Leveraging {OOD} data for proactive backdoor detection in federated learning," in *33rd USENIX Security Symposium (USENIX Security 24)*, pp. 4193–4210, 2024.
15. Z. Ma and T. Gao, "Federated learning backdoor attack detection with persistence diagram," *Computers & Security*, vol. 136, p. 103557, 2024.
16. K. Zhang, G. Tao, Q. Xu, S. Cheng, S. An, Y. Liu, S. Feng, G. Shen, P. Chen, S. Ma, and X. Zhang, "FLIP: A provable defense framework for backdoor mitigation in federated learning," in *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*, OpenReview.net, 2023.
17. Z. Cai, J. Cai, P. Lian, X. Liu, Y. Che, *et al.*, "Fedpd: Defending federated prototype learning against backdoor attacks," *Neural Networks*, vol. 184, p. 107016, 2025.
18. I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," in *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings* (Y. Bengio and Y. LeCun, eds.), 2015.
19. B. Mu, Z. Niu, L. Wang, X. Wang, Q. Miao, R. Jin, and G. Hua, "Progressive backdoor erasing via connecting backdoor and adversarial attacks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 20495–20503, 2023.
20. J.-L. Yin, W. Wang, W. Lin, X. Liu, *et al.*, "Adversarial-inspired backdoor defense via bridging backdoor and adversarial attacks," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, pp. 9508–9516, 2025.
21. A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," in *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*, OpenReview.net, 2018.
22. D. Yin, Y. Chen, R. Kannan, and P. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates," in *International conference on machine learning*, pp. 5650–5659, Pmlr, 2018.
23. R. Guerraoui, S. Rouault, *et al.*, "The hidden vulnerability of distributed learning in byzantium," in *International conference on machine learning*, pp. 3521–3530, PMLR, 2018.
24. P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," *Advances in neural information processing systems*, vol. 30, 2017.